Автономная некоммерческая организация высшего образования
«Санкт-Петербургский институт экономики и управления»



Кафедра 09.02.07

ПРАКТИЧЕСКАЯ РАБОТА № 7

Дисциплина: МДК 03.01

Тема: Внедрение и приёмочные испытания

Вариант № 2

Студент: Шваб Роберт Эдуардович Подпись _____

Группа: И13_24_11

Руководитель: _____ Подпись _____

Дата представления: «___» _____ 2026 г.

Оценка: _____

Санкт-Петербург 2026

СОДЕРЖАНИЕ

1 ВЫПОЛНЕНИЕ ПРАКТИЧЕСКОЙ РАБОТЫ.....3
1.1 Цель и исходные данные.....3
1.2 План перехода.....3
1.3 Критерии испытаний.....4
1.4 Программа и воспроизведение результата.....5
1.5 Выводы.....5
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ.....6
ПРИЛОЖЕНИЕ А.....7
ПРИЛОЖЕНИЕ Б.....10

1 ВЫПОЛНЕНИЕ ПРАКТИЧЕСКОЙ РАБОТЫ

1.1 Цель и исходные данные

Цель работы — составить дорожную карту перехода на систему заявок, определить критерии FAT/SAT и план обучения пользователей.

Постановка работы приведена в перечне заданий МДК 03.01 [1].

Исходники и отчёты: репозиторий MDK0301-ServiceDesk на GitHub

1.2 План перехода

Заказчик в учебном сценарии — служба технической поддержки на 20 сотрудников. До ввода администратор проверяет доступ, снимает резервную копию и фиксирует конфигурацию. Этапы и условия перехода даны в таблице 1 и на рисунке 1.

Таблица 1 — Этапы перехода и критерии выхода

Срок	Работа	Критерий выхода
Дни 1–2	Подготовка стенда	Доступ и пробное восстановление
День 3	FAT	Контракты, загрузка и миграция проверены
Дни 4–5	Обучение и пилот	5 учебных заявок обработано
День 6	SAT	Критические сценарии пройдены
День 7	Go-Live	Нет P1 в течение первых 24 часов

Источник: составлено по материалам практической работы.

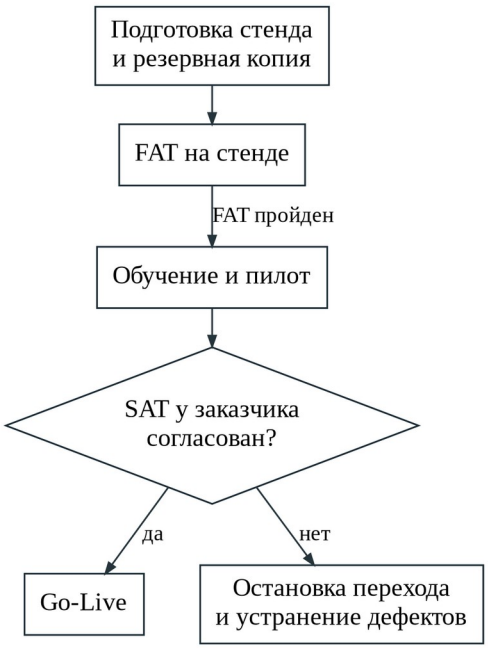


Рисунок 1 — План перехода на новую систему

Обучение оператора (1 час): создать заявку, назначить приоритет, передать на L2, закрыть после подтверждения. Обучение администратора (1 час): запустить API, прочитать журнал, выполнить SAT и восстановить резервную копию в отдельную папку. Результат обучения — пять обработанных учебных заявок без потери данных.

Go-Live разрешается после выполнения критических сценариев и согласования протокола ответственными. При недоступности v1, потере данных или P1 переключение останавливается; возвращается прежняя версия на совместимой схеме, заявки сверяются с резервной копией.

1.3 Критерии испытаний

По постановке [1] ориентиром для видов испытаний выступает ГОСТ 34.603-92. Для действующей российской нормативной базы отдельно учитывается ГОСТ Р 59792-2021 [2]. FAT проводится на учебном стенде; SAT у заказчика предусматривает рабочее окружение, контрольные данные, протокол и согласование.

Программа SAT проверяет /health, списки v1 и v2 и ответ 404 для отсутствующей карточки. Протокол SAT.md сохраняется в RunResults.

Фактические четыре HTTP-ответа и итог 4/4 PASS приведены в приложении Б.

1.4 Программа и воспроизведение результата

Программа находится в Code/Operations/Program.cs, метод Sat. Профиль «ПР 7» запускает API, проверяет фактические HTTP-ответы, печатает каждый результат и формирует SAT.md. Реализация на C# приведена в приложении А.

Перед показом выполняется «Сборка → Пересобрать решение». Результат сборки виден в окне «Вывод → Сборка», а результат программы — в отдельной консоли Visual Studio.

1.5 Выводы

План ввода задаёт условия перехода и остановки. Автоматическая часть SAT повторяется в Visual Studio через профиль «ПР 7» проекта Verification.

Листинги приведены в приложении А.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1 МДК 03.01. Перечень практических работ № 1–8. Учебное задание.

2 Росстандарт. ГОСТ Р 59792-2021. Виды испытаний автоматизированных систем. https://protect.gost.ru/gost/details/60279c95-21dc-430c-bf2c-f626c588576e

ПРИЛОЖЕНИЕ А

Исходный код программы

Code/Operations/Program.cs

```
using System.IO.Compression;
using System.Net;
using System.Security.Cryptography;
using System.Text.Json;
using Microsoft.Data.Sqlite;

try
{
    switch (args)
    {
        case ["manifest", var version, var file, var output]:
            File.WriteAllText(output, JsonSerializer.Serialize(new { Version = version, Sha256 = Hash(file) }));
            Console.WriteLine("Manifest written: " + output);
            return 0;
        case ["backup", var source, var destination]:
            if
(Path.GetFullPath(destination).StartsWith(Path.TrimEndingDirectorySeparator(Path.GetFullPath(source)) +
Path.DirectorySeparatorChar,
                OperatingSystem.IsWindows() ? StringComparison.OrdinalIgnoreCase : StringComparison.Ordinal))
                throw new ArgumentException("Archive must be outside source directory");
            ZipFile.CreateFromDirectory(source, destination);
            File.WriteAllText(destination + ".sha256", Hash(destination));
            using (var zip = ZipFile.OpenRead(destination))
                Console.WriteLine($"Backup: {zip.Entries.Count} entries; SHA-256 {Hash(destination)}");
            return 0;
        case ["sat", var address, var output]:
            return await Sat(address, output);
        case ["migration", var directory, var database]:
            return Migration(directory, database);
        default:
            Console.Error.WriteLine("Operations: manifest VERSION FILE JSON | backup SOURCE ZIP | sat URL
MARKDOWN | migration SQL_DIR DB");
            return 1;
    }
}
catch (Exception error)
{
    Console.Error.WriteLine(error.Message);
    return 1;
}

static string Hash(string path)
{
    using var stream = File.OpenRead(path);
    return Convert.ToHexString(SHA256.HashData(stream));
}
static async Task<int> Sat(string address, string output)
{
    using var http = new HttpClient { BaseAddress = new Uri(address), Timeout = TimeSpan.FromSeconds(5) };
    var checks = new List<(string Scenario, bool Passed, string Detail)>();
    foreach (var (route, expected) in new[]
    {
```

```
    ("/health", HttpStatusCode.OK),
    ("/api/v1/tickets", HttpStatusCode.OK),
    ("/api/v2/tickets", HttpStatusCode.OK),
    ("/api/v1/tickets/2147483647", HttpStatusCode.NotFound)
  })
  {
    try
    {
      using var response = await http.GetAsync(route);
      var correctBody = true;
      if (expected != HttpStatusCode.NotFound)
      {
        using var json = JsonDocument.Parse(await response.Content.ReadAsStringAsync());
        correctBody = route == "/health" ? json.RootElement.GetProperty("status").GetString() == "ok"
          : json.RootElement.ValueKind == JsonValueKind.Array;
      }
      checks.Add((route, response.StatusCode == expected && correctBody, $"HTTP
{(int)response.StatusCode}"));
    }
    catch (Exception e) when (e is HttpRequestException or TaskCanceledException or JsonException or
KeyNotFoundException or InvalidOperationException)
    { checks.Add((route, false, e.GetType().Name)); }
  }
  foreach (var x in checks) Console.WriteLine($"{(x.Passed ? "PASS" : "FAIL")}: GET {x.Scenario} —
{x.Detail}");
  var passed = checks.Count(x => x.Passed);
  var lines = new List<string> { "# Протокол SAT — стенд", "", $"Дата: {DateTimeOffset.UtcNow:yyyy-MM-
dd HH:mm:ss} UTC", "",
    "| Сценарий | Результат | Ответ |", "| --- | --- | --- |" };
  foreach (var x in checks) lines.Add($"| GET {x.Scenario} | {(x.Passed ? "PASS" : "FAIL")} | {x.Detail} |");
  lines.Add($"\nПройдено: {passed}/{checks.Count}. Согласование заказчиком проводится отдельно.");
  File.WriteAllLines(output, lines);
  Console.WriteLine($"SAT: {passed}/{checks.Count} PASS");
  return passed == checks.Count ? 0 : 2;
}
static int Migration(string directory, string database)
{
  if (File.Exists(database)) throw new ArgumentException("Use a new database file for the migration
demo");
  using var connection = new SqliteConnection(new SqliteConnectionStringBuilder { DataSource =
database }.ToString());
  connection.Open();
  void Exec(string sql) { using var cmd = connection.CreateCommand(); cmd.CommandText = sql;
cmd.ExecuteNonQuery(); }
  long Read(string sql) { using var cmd = connection.CreateCommand(); cmd.CommandText = sql; return
Convert.ToInt64(cmd.ExecuteScalar()); }
  Exec(File.ReadAllText(Path.Combine(directory, "schema_v1.sql")));
  Exec("INSERT INTO tickets (title,status,legacy_category) VALUES ('Existing ticket','Open','Hardware')");
  Exec(File.ReadAllText(Path.Combine(directory, "migrate_expand.sql")));
  Exec("INSERT INTO tickets (title,status) VALUES ('Old client ticket','Open')");
  if (Read("SELECT COUNT(*) FROM tickets WHERE priority=3") != 2) throw new
InvalidOperationException("Expand failed");
  Console.WriteLine("PASS: original row preserved and v1 insert works after Expand");
  Console.WriteLine("After Expand: 2 rows, priority=3; INSERT v1 works.");
  Exec(File.ReadAllText(Path.Combine(directory, "migrate_contract.sql")));
  if (Read("SELECT COUNT(*) FROM tickets WHERE priority=3") != 2 ||
    Read("SELECT COUNT(*) FROM pragma_table_info('tickets') WHERE name='legacy_category'") != 0)
    throw new InvalidOperationException("Contract failed");
  Console.WriteLine("PASS: Contract removes the legacy column and keeps both rows");
```

```
    Console.WriteLine("After Contract: 2 rows; legacy_category absent.");
    return 0;
}
```

Code/Operations/Operations.csproj

```
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup><OutputType>Exe</OutputType><TargetFramework>net8.0</
TargetFramework><ImplicitUsings>enable</ImplicitUsings><Nullable>enable</Nullable></PropertyGroup>
  <ItemGroup><PackageReference Include="Microsoft.Data.Sqlite" Version="8.0.11"/></ItemGroup>
</Project>
```

ПРИЛОЖЕНИЕ Б

Фактический вывод контрольного прогона

```
PASS: GET /health — HTTP 200
PASS: GET /api/v1/tickets — HTTP 200
PASS: GET /api/v2/tickets — HTTP 200
PASS: GET /api/v1/tickets/2147483647 — HTTP 404
SAT: 4/4 PASS
```

Снимки выполнения программы в Visual Studio

Я запустил программы в Visual Studio под Windows. На снимках приведены исходный код и результаты контрольного запуска.

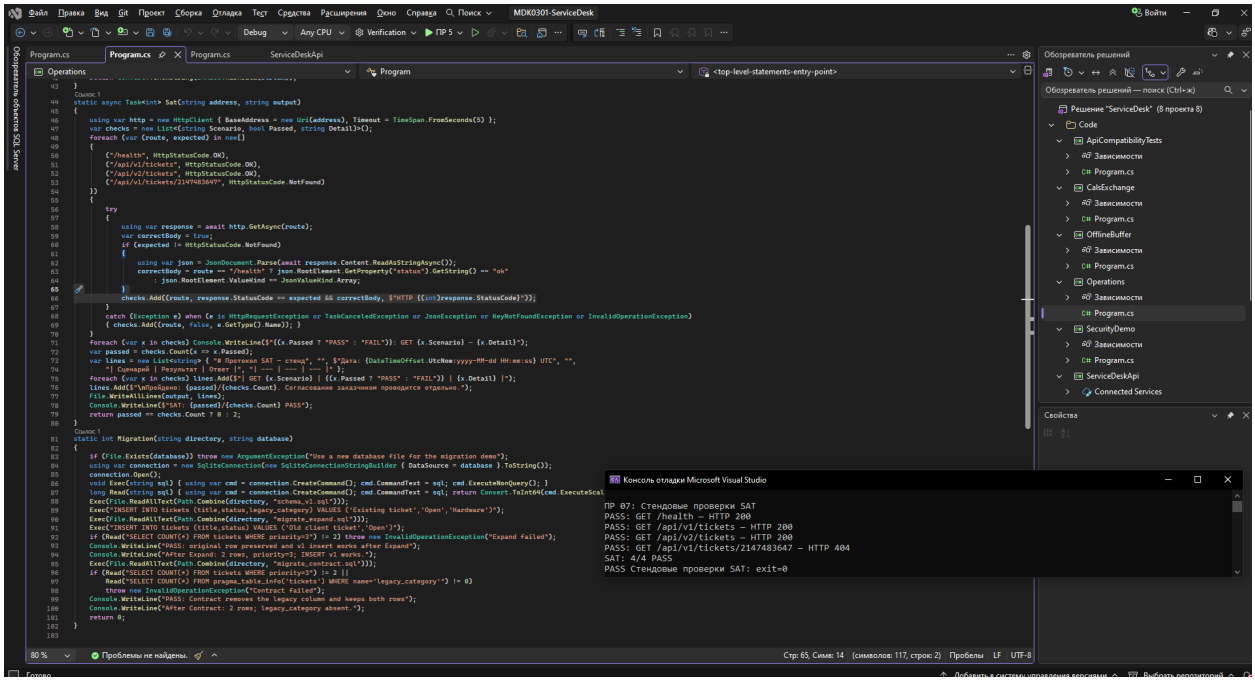


Рисунок Б.1 — Стендовые проверки SAT. Получены ожидаемые ответы HTTP 200 и HTTP 404; пройдены четыре проверки.